# MySQL Interview Preparation Guide

60 Questions & Answers

Fundamentals · Constraints · Queries & Joins · Aggregation · Window Functions · Indexes & Transactions

## Questions 1–10: MySQL Fundamentals

### Q1 What is MySQL?

ANSWER

MySQL is an open-source Relational Database Management System (RDBMS) used to store, manage, and retrieve structured data using SQL. It organises data into tables made up of rows and columns.

EXAMPLE — CUSTOMER TABLE

| customer_id | customer_name | city |
|---|---|---|
| 101 | Deepraj | Hyderabad |
| 102 | Ravi | Bangalore |

KEY FEATURES

- Open-source and free
- Fast and reliable
- Supports ACID transactions
- Multi-user concurrent access
- Highly scalable

### Q2 What is the difference between SQL and MySQL?

ANSWER

| | SQL | MySQL |
|---|---|---|
| What it is | A query language | A database software product |
| Purpose | Interact with databases | Manages data using SQL |
| Creator | ANSI standard | Oracle Corporation |

> *Analogy: SQL is the language. MySQL is the software that understands and speaks it.*

### Q3 What are the types of SQL commands?

ANSWER

| Type | Full Form | Commands |
|---|---|---|
| DDL | Data Definition Language | CREATE, ALTER, DROP, TRUNCATE |
| DML | Data Manipulation Language | INSERT, UPDATE, DELETE |
| DQL | Data Query Language | SELECT |
| TCL | Transaction Control Language | COMMIT, ROLLBACK |
| DCL | Data Control Language | GRANT, REVOKE |

## Q4 What is a database?

ANSWER

A database is an organised collection of related data stored electronically for easy retrieval and management — like a digital filing cabinet.

> *Example: A college database stores Students, Courses, Faculty, and Marks — all interrelated.*

## Q5 What is a table?

ANSWER

A table is a structured collection of data arranged in rows (records) and columns (fields/attributes).

| emp_id | name | salary |
|---|---|---|
| 1 | John | 50,000 |
| 2 | Priya | 65,000 |

## Q6 What is a Primary Key?

ANSWER

A primary key uniquely identifies each row in a table.

PROPERTIES

- Must be UNIQUE across all rows
- Cannot be NULL
- Only one primary key per table

> *Real-World: In an online shopping system, customer_id uniquely identifies each customer.*

## Q7 What is a Foreign Key?

ANSWER

A foreign key is a column that links two tables by referencing the primary key of another table. It establishes a parent-child relationship.

| Table | Key Column | Role |
|---|---|---|
| CUSTOMERS | customer_id | Primary Key (parent) |
| ORDERS | customer_id | Foreign Key (child) — references CUSTOMERS |

> *Real-World: One customer can place many orders. Each order's customer_id must match an existing customer.*

## Q8 Primary Key vs Foreign Key

COMPARISON

| Aspect | Primary Key | Foreign Key |
|---|---|---|
| Purpose | Uniquely identifies each row | Links tables together |
| NULL allowed? | No | Yes (unless constrained) |
| Count per table | Only one | Multiple allowed |
| Integrity enforced | Entity integrity | Referential integrity |

## Q9 DELETE vs TRUNCATE vs DROP

COMPARISON

| Command | Removes Data? | Removes Structure? | WHERE Allowed? | Rollback? |
|---|---|---|---|---|
| DELETE | Yes (selected rows) | No | Yes | Yes |
| TRUNCATE | Yes (all rows) | No | No | No |
| DROP | Yes (everything) | Yes | No | No |

```
DELETE FROM employees WHERE emp_id = 1;
TRUNCATE TABLE employees;
DROP TABLE employees;
```

> *Use DELETE for selective removal, TRUNCATE to clear staging tables before a reload, DROP to remove an obsolete table entirely.*

## Q10 CHAR vs VARCHAR

COMPARISON

| Aspect | CHAR | VARCHAR |
|---|---|---|
| Storage | Fixed length — pads unused space | Variable length — stores only actual data |

| | | |
|---|---|---|
| Speed | Faster for fixed-size values | Saves storage for variable data |
| Example | CHAR(10) storing 'INDIA' uses 10 chars | VARCHAR(10) storing 'INDIA' uses 5 chars |

> *Use CHAR for fixed codes like country codes (IN, US) or gender flags (M, F). Use VARCHAR for names, emails, and addresses.*

## Questions 11–20: Constraints & NULL Handling

### Q11 What are constraints?

**ANSWER**

Constraints are rules applied to columns to enforce data accuracy and integrity. They prevent invalid data from being inserted.

| Constraint | Purpose |
|---|---|
| PRIMARY KEY | Uniquely identifies each row |
| FOREIGN KEY | Maintains relationships between tables |
| UNIQUE | Prevents duplicate values |
| NOT NULL | Disallows NULL values |
| DEFAULT | Assigns a default value when none is provided |
| CHECK | Enforces a custom condition (e.g. price > 0) |
| AUTO_INCREMENT | Generates sequential numbers automatically |

### Q12 What is the NOT NULL constraint?

**ANSWER**

NOT NULL ensures a column must always contain a value — it cannot be left empty.

```
CREATE TABLE students (
    student_id   INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL
);
```

> *Scenario: A company's system must always capture an employee name — blank records are not allowed.*

### Q13 What is the UNIQUE constraint — and how does it differ from PRIMARY KEY?

ANSWER

UNIQUE ensures no two rows have the same value in the specified column.

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    email   VARCHAR(100) UNIQUE
);
```

| Aspect | PRIMARY KEY | UNIQUE |
|---|---|---|
| NULL allowed? | Never | One NULL allowed in MySQL |
| Count per table | Only one | Multiple UNIQUE columns allowed |

## Q14 What is the DEFAULT constraint?

ANSWER

DEFAULT automatically assigns a value when no value is provided during INSERT.

```
CREATE TABLE orders (
    order_id     INT PRIMARY KEY,
    order_status VARCHAR(20) DEFAULT 'Pending'
);
```

> *Scenario: If order status is not explicitly set, it defaults to 'Pending' automatically.*

## Q15 What is AUTO_INCREMENT?

ANSWER

AUTO_INCREMENT automatically generates a unique, sequentially increasing numeric value for each new row — typically used with primary keys.

```
CREATE TABLE employees (
    emp_id   INT AUTO_INCREMENT PRIMARY KEY,
    emp_name VARCHAR(100)
);
```

> *Scenario: Every new employee automatically receives a unique employee ID — no manual assignment needed.*

## Q16 What is the CHECK constraint?

ANSWER

CHECK validates data against a custom condition before allowing an INSERT or UPDATE.

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    price      DECIMAL(10,2),
    CHECK (price > 0)
);
```

> *Scenario: Products cannot have a zero or negative price. The CHECK constraint rejects such values at the database level.*

## Q17 What is NULL in MySQL?

ANSWER

NULL represents missing or unknown information — it means no value exists for this field.

IMPORTANT — NULL IS NOT:

- Zero (0) — that is a numeric value
- Empty string ('') — that is a known value with no characters
- False — that is a boolean value

## Q18 NULL vs Empty String ('')

COMPARISON

| Aspect | NULL | Empty String '' |
|---|---|---|
| Meaning | Unknown or missing value | Known value with no characters |
| Length | No concept of length | Length = 0 |
| Check with | IS NULL / IS NOT NULL | = '' or = "" |

> *Example: NULL → customer did not provide email. '' → customer intentionally stored a blank value.*

## Q19 What is the execution order of a SELECT query?

ANSWER

Although you write SELECT first, MySQL processes clauses in this order:

| Step | Clause | What It Does |
|---|---|---|
| 1 | FROM | Identify source tables |
| 2 | JOIN | Combine tables |
| 3 | WHERE | Filter rows before grouping |
| 4 | GROUP BY | Group filtered rows |
| 5 | HAVING | Filter groups using aggregates |
| 6 | SELECT | Choose output columns |
| 7 | DISTINCT | Remove duplicates |
| 8 | ORDER BY | Sort results |
| 9 | LIMIT | Restrict row count |

> *Key insight: You cannot use aggregate functions directly in WHERE — it runs before GROUP BY. Use HAVING instead.*

### Q20 WHERE vs HAVING

COMPARISON

| Aspect | WHERE | HAVING |
|---|---|---|
| When it runs | Before GROUP BY | After GROUP BY |
| Works with | Row-level conditions | Group-level / aggregate conditions |
| Aggregate functions? | Not directly | Yes |

```
SELECT department, COUNT(*)
FROM employees
WHERE  salary > 30000      -- filter rows first
GROUP BY department
HAVING COUNT(*) > 5;       -- then filter groups
```

## Questions 21–30: Aggregation, Sorting & Conditional Logic

### Q21 What is GROUP BY?

ANSWER

GROUP BY combines rows sharing the same value in specified columns into a single group, so aggregate functions can be applied per group.

```
SELECT   department, AVG(salary) AS avg_salary
FROM     employees
GROUP BY department;
```

> *Scenario: HR wants the average salary per department — not a single average for the whole company.*

### Q22 What is ORDER BY?

ANSWER

ORDER BY sorts the result set in ascending (ASC, default) or descending (DESC) order based on one or more columns.

```
SELECT product_name, price
FROM   products
ORDER BY price DESC;   -- most expensive first
```

### Q23 What is LIMIT?

ANSWER

LIMIT restricts the number of rows returned. Commonly paired with ORDER BY for top-N queries.

```
SELECT * FROM orders
ORDER BY order_date DESC
LIMIT 10;   -- latest 10 orders only
```

> *Use LIMIT with OFFSET for pagination: LIMIT 10 OFFSET 20 skips the first 20 rows and returns the next 10.*

## Q24 What are aggregate functions?

**ANSWER**

Aggregate functions summarise multiple rows into a single result value.

| Function | Description |
|---|---|
| COUNT() | Counts rows |
| SUM() | Adds all values |
| AVG() | Calculates average |
| MIN() | Finds the smallest value |
| MAX() | Finds the largest value |

```
SELECT COUNT(*)          AS total_orders,
       SUM(order_amount) AS total_revenue
FROM   orders;
```

## Q25 What is DISTINCT?

**ANSWER**

DISTINCT eliminates duplicate values from the result set, returning only unique values.

```
SELECT DISTINCT city FROM customers;
```

## Q26 What are column aliases?

**ANSWER**

An alias gives a temporary, more readable name to a column or expression in the result set using the AS keyword.

```
SELECT SUM(sales_amount) AS total_sales,
       AVG(sales_amount) AS avg_sales
FROM   sales;
```

> *Aliases do not change the actual column name — they only apply to the query output. Useful in reports and dashboards.*

## Q27 What is CASE WHEN?

**ANSWER**

CASE WHEN applies if-else conditional logic within a query, allowing you to derive new values based on column conditions.

```
SELECT employee_name, salary,
       CASE
           WHEN salary >= 100000 THEN 'High'
           WHEN salary >= 50000  THEN 'Medium'
           ELSE                       'Low'
       END AS salary_category
FROM employees;
```

> *Scenario: Categorise employees into salary bands for reporting — without changing any underlying data.*

## Q28 What is IFNULL()?

**ANSWER**

IFNULL(expression, replacement) returns the replacement value if the expression evaluates to NULL, otherwise returns the expression.

```
SELECT customer_name,
       IFNULL(phone_number, 'Not Provided') AS phone
FROM   customers;
```

## Q29 COALESCE() vs IFNULL()

**ANSWER**

COALESCE() returns the first non-NULL value from a list of expressions. It is more flexible than IFNULL().

```
SELECT customer_name,
       COALESCE(mobile, home_phone, office_phone, 'No Contact') AS contact
FROM   customers;
```

| Function | Expressions Allowed |
|---|---|
| IFNULL() | Exactly 2 |
| COALESCE() | 2 or more |

## Q30 What are common date functions?

**ANSWER**

| Function | Purpose |
|---|---|
| CURDATE() | Returns today's date |
| NOW() | Returns current date and time |
| YEAR() / MONTH() | Extract year or month from a date |
| DATEDIFF() | Difference in days between two dates |

| | |
|---|---|
| DATE_ADD() | Add an interval to a date |
| DATE_SUB() | Subtract an interval from a date |

```
SELECT order_id,
       DATEDIFF(CURDATE(), order_date) AS days_since_order
FROM   orders;
```

## Questions 31–40: Joins, Set Operations & Subqueries

### Q31 What is INNER JOIN?

**ANSWER**

INNER JOIN returns only rows that have matching values in both tables. Rows without a match are excluded.

```
SELECT c.customer_name, o.order_id
FROM   customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

> *Scenario: Show only customers who have placed at least one order.*

### Q32 What is LEFT JOIN?

**ANSWER**

LEFT JOIN returns all rows from the left table plus matching rows from the right table. Where no match exists, right-side columns are NULL.

```
SELECT c.customer_name, o.order_id
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
-- Customers with no orders appear with order_id = NULL
```

> *Scenario: List ALL customers, including those who have never placed an order.*

### Q33 What is RIGHT JOIN?

**ANSWER**

RIGHT JOIN returns all rows from the right table plus matching rows from the left table. Where no match exists, left-side columns are NULL.

```
SELECT c.customer_name, o.order_id
FROM   customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

> *Most developers rewrite RIGHT JOIN as a LEFT JOIN by swapping table order for clarity.*

## Q34 What is a SELF JOIN?

ANSWER

A SELF JOIN joins a table with itself. Useful for hierarchical data — for example, finding the manager of each employee stored in the same table.

```
SELECT e.employee_name,
       m.employee_name AS manager_name
FROM   employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

> *Always use aliases (e, m) when doing a SELF JOIN to distinguish the two references of the same table.*

## Q35 What is CROSS JOIN?

ANSWER

CROSS JOIN returns the Cartesian product — every row from the first table paired with every row from the second.

```
SELECT s.size, c.color
FROM   sizes s
CROSS JOIN colors c;   -- 3 sizes × 4 colors = 12 rows
```

> *Scenario: Generate all possible combinations of product sizes and colours. Use with caution on large tables — row count multiplies rapidly.*

## Q36 UNION vs UNION ALL

COMPARISON

| Aspect | UNION | UNION ALL |
|---|---|---|
| Duplicates | Removes duplicates | Keeps all rows including duplicates |
| Performance | Slower (deduplication step) | Faster |
| Use when | You need distinct combined results | You know there are no duplicates or need all rows |

```
SELECT city FROM customers UNION SELECT city FROM suppliers;
SELECT city FROM customers UNION ALL SELECT city FROM suppliers;
```

## Q37 What is a subquery?

ANSWER

A subquery is a SELECT query nested inside another query. The inner query runs first and its result is used by the outer query.

```
SELECT employee_name, salary
FROM   employees
WHERE  salary > (
```

```
    SELECT AVG(salary) FROM employees
);
```

> *Scenario: Find all employees earning above the company-wide average salary.*

## Q38 What is a correlated subquery?

ANSWER

A correlated subquery references a column from the outer query. It executes once for each row in the outer query.

```
SELECT e1.employee_name, e1.salary, e1.department
FROM   employees e1
WHERE  salary > (
    SELECT AVG(e2.salary)
    FROM   employees e2
    WHERE  e2.department = e1.department   -- references outer row
);
```

> *Scenario: Find employees earning more than their own department's average — the subquery recalculates for each department.*

## Q39 EXISTS vs IN

COMPARISON

| Aspect | IN | EXISTS |
|---|---|---|
| What it does | Checks value against a list | Checks whether matching rows exist |
| Best for | Small result sets | Large subqueries |
| Returns | Values | TRUE or FALSE |

```
SELECT c.customer_name FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE  o.customer_id = c.customer_id
);
```

## Q40 What is a CTE (Common Table Expression)?

ANSWER

A CTE uses the WITH clause to create a temporary named result set that can be referenced within the same query. It improves readability of complex queries.

```
WITH dept_salary AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM   employees
    GROUP BY department
)
SELECT * FROM dept_salary WHERE avg_salary > 60000;
```

BENEFITS

- Improves readability by naming intermediate results
- Breaks complex queries into logical steps
- Easier to debug than deeply nested subqueries

## Questions 41–50: Window Functions & Database Objects

### Q41 What are window functions?

ANSWER

Window functions perform calculations across a related set of rows while keeping each individual row visible — unlike GROUP BY, which collapses rows into groups.

```
SELECT employee_name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM   employees;
-- Each employee row is preserved AND shows the department average
```

> *Key difference from GROUP BY: window functions do not reduce the number of rows in the result.*

### Q42 What is ROW_NUMBER()?

ANSWER

ROW_NUMBER() assigns a unique sequential integer to each row based on the ORDER BY clause. No two rows get the same number, even if values are tied.

```
SELECT employee_name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM   employees;
```

### Q43 ROW_NUMBER() vs RANK() vs DENSE_RANK()

COMPARISON — GIVEN SALARIES: 100, 90, 90, 80

| Salary | ROW_NUMBER() | RANK() | DENSE_RANK() |
|---|---|---|---|
| 100 | 1 | 1 | 1 |
| 90 | 2 | 2 | 2 |
| 90 | 3 | 2 (tie) | 2 (tie) |
| 80 | 4 | 4 (skips 3) | 3 (no skip) |

| Function | Behaviour on Ties |
|---|---|
| ROW_NUMBER() | Always unique — arbitrary ordering for tied values |
| RANK() | Same rank for ties, then skips the next rank(s) |

| | |
|---|---|
| DENSE_RANK() | Same rank for ties, no ranks are skipped |

## Q44 What is LEAD()?

**ANSWER**

LEAD() accesses data from the next row in the ordered result set, without requiring a self-join.

```
SELECT order_date, sales_amount,
       LEAD(sales_amount) OVER (ORDER BY order_date) AS next_day_sales
FROM   sales;
```

> *Use Case: Compare today's sales against the next day's to detect trends.*

## Q45 What is LAG()?

**ANSWER**

LAG() accesses data from the previous row in the ordered result set — the mirror of LEAD().

```
SELECT order_date, sales_amount,
       LAG(sales_amount) OVER (ORDER BY order_date) AS prev_day_sales
FROM   sales;
```

> *Use Case: Calculate day-over-day change: sales_amount - LAG(sales_amount) OVER (ORDER BY order_date).*

## Q46 How do you calculate a running total?

**ANSWER**

Use SUM() as a window function with ORDER BY to accumulate values progressively across rows.

```
SELECT order_date, sales_amount,
       SUM(sales_amount) OVER (ORDER BY order_date) AS running_total
FROM   sales;
```

> *Add PARTITION BY to reset the running total per group — e.g. per month or per region.*

## Q47 What is a View?

**ANSWER**

A view is a virtual table defined by a stored SELECT query. It does not store data — data is fetched from the underlying tables each time the view is queried.

```
CREATE VIEW high_salary_employees AS
SELECT employee_name, salary
FROM   employees WHERE salary > 100000;

SELECT * FROM high_salary_employees;
```

**BENEFITS**

- Simplifies complex queries by giving them a reusable name

- Improves security — expose only specific columns or rows
- Acts as an abstraction layer over the underlying tables

## Q48 What is a Stored Procedure?

ANSWER

A stored procedure is a reusable, named block of SQL code saved in the database and executed on demand. It can accept input parameters and contain logic.

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM customers;
END //
DELIMITER ;

CALL GetAllCustomers();
```

> *Benefits: Reduces repeated SQL code, improves performance (pre-compiled), and allows access control — grant EXECUTE without exposing underlying tables.*

## Q49 What is a Trigger?

ANSWER

A trigger is a stored procedure that automatically executes when an INSERT, UPDATE, or DELETE event occurs on a table.

```
CREATE TRIGGER before_product_insert
BEFORE INSERT ON products
FOR EACH ROW
SET NEW.created_at = NOW();
```

| Timing | Events |
|---|---|
| BEFORE | INSERT, UPDATE, DELETE |
| AFTER | INSERT, UPDATE, DELETE |

> *Common uses: audit logging, auto-populating timestamps, enforcing business rules at the database level.*

## Q50 What are transactions and ACID properties?

ANSWER

A transaction is a group of SQL operations treated as a single unit — all succeed together, or all fail together.

| Property | Meaning |
|---|---|
| Atomicity | All operations succeed or all fail — no partial updates |

| | |
|---|---|
| Consistency | Data remains valid before and after the transaction |
| Isolation | Concurrent transactions do not interfere with each other |
| Durability | Committed changes survive system crashes |

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;
COMMIT;   -- or ROLLBACK if something goes wrong
```

> *Real-World: Bank transfer — both the debit and credit must succeed together. If either fails, everything is rolled back.*

## Questions 51–60: Indexes, Optimization & Concurrency

### Q51 What is an index?

**ANSWER**

An index is a data structure that lets MySQL locate rows quickly without scanning every row in the table — similar to a book's index pointing you directly to the right page.

```
CREATE INDEX idx_customer_email ON customers(email);
```

| Benefits | Trade-Offs |
|---|---|
| Faster SELECT queries | Consumes additional disk space |
| Improves WHERE, JOIN, ORDER BY | Slows INSERT, UPDATE, DELETE |
| Essential on large tables | Too many indexes hurt write performance |

### Q52 What are the types of indexes?

**ANSWER**

| Index Type | Purpose |
|---|---|
| Primary Index | Created automatically for PRIMARY KEY |
| Unique Index | Prevents duplicate values |
| Composite Index | Covers multiple columns |
| Full-Text Index | Optimised for text-based search |
| Spatial Index | Used for geographic/geometry data |

### Q53 What is a composite index?

**ANSWER**

A composite index is built on two or more columns. Most effective when queries filter on those columns together.

```
CREATE INDEX idx_city_status ON orders(city, order_status);

SELECT * FROM orders WHERE city = 'Hyderabad' AND order_status = 'Shipped';
```

### Q54 Why does column order matter in a composite index?

ANSWER

MySQL uses the leftmost prefix rule — it can only use a composite index if the query filters on the leading columns first.

| Query | Uses Index on (department, salary)? |
|---|---|
| WHERE department = 'IT' | Yes — uses first column |
| WHERE department = 'IT' AND salary > 50000 | Yes — uses both columns |
| WHERE salary > 50000 | No — skips the first column |

> *Design tip: Put the most selective (highest cardinality) column first in a composite index.*

### Q55 What is query optimization?

ANSWER

Query optimisation is the process of rewriting or restructuring SQL to reduce CPU, memory, and disk usage, improving execution speed.

COMMON TECHNIQUES

- Add indexes on frequently filtered, joined, or sorted columns
- Avoid SELECT * — select only the columns you need
- Filter early with WHERE to reduce rows before joins or grouping
- Avoid functions on indexed columns in WHERE — it prevents index use
- Use JOINs instead of correlated subqueries where possible
- Use LIMIT on large result sets
- Analyse slow queries using EXPLAIN

### Q56 What is EXPLAIN?

ANSWER

EXPLAIN shows the query execution plan — how MySQL will process a query, which indexes it uses, and how many rows it estimates scanning.

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;
```

| Column | What to Look For |
|---|---|
| type | 'ref' or 'eq_ref' is good. 'ALL' means full table scan — investigate |

| | |
|---|---|
| key | Which index MySQL chose. NULL means no index used |
| rows | Estimated rows examined. Lower is better |
| Extra | 'Using index' is good. 'Using filesort' or 'Using temporary' may need tuning |

## Q57 What is normalization?

ANSWER

Normalization organises a database to reduce data redundancy and improve integrity by splitting large tables into smaller, related ones.

| Normal Form | Rule |
|---|---|
| 1NF | Each column holds atomic (indivisible) values; no repeating groups |
| 2NF | 1NF + every non-key column depends on the entire primary key |
| 3NF | 2NF + no non-key column depends on another non-key column |
| BCNF | Every determinant must be a candidate key |

> *Example: Instead of repeating customer name and city in every order row, store customer details once in a CUSTOMERS table and reference via customer_id.*

## Q58 What is denormalization?

ANSWER

Denormalization intentionally introduces redundancy to improve read performance — accepting some data duplication to avoid expensive joins.

| Aspect | Details |
|---|---|
| When to use | Read-heavy reporting tables, data warehouses, dashboards |
| Benefit | Faster reads — fewer JOINs |
| Trade-off | Increased storage; consistency harder to maintain on updates |

> *Common pattern: OLTP systems stay normalised; OLAP/reporting systems are often denormalised.*

## Q59 What is a deadlock?

ANSWER

A deadlock occurs when two transactions each hold a lock on a resource the other needs — both wait indefinitely and neither can proceed.

| | Transaction A | Transaction B |
|---|---|---|
| Step 1 | Locks Row 1 | Locks Row 2 |

| | | |
|---|---|---|
| Step 2 | Waiting for Row 2 | Waiting for Row 1 |
| Result | Deadlock — neither can proceed | |

MYSQL BEHAVIOUR

InnoDB automatically detects deadlocks and rolls back the transaction with the fewest modified rows to break the cycle.

PREVENTION TIPS

- Always access tables and rows in the same order across transactions
- Keep transactions short and commit quickly
- Use SELECT ... FOR UPDATE only when necessary

## Q60 What are transaction isolation levels?

ANSWER

Isolation levels control how much a transaction is affected by concurrent transactions. Higher isolation means more consistency but lower concurrency.

| Isolation Level | Dirty Read | Non-Repeatable Read | Phantom Read |
|---|---|---|---|
| READ UNCOMMITTED | Possible | Possible | Possible |
| READ COMMITTED | Prevented | Possible | Possible |
| REPEATABLE READ (default) | Prevented | Prevented | Possible |
| SERIALIZABLE | Prevented | Prevented | Prevented |

| Phenomenon | Description |
|---|---|
| Dirty Read | Reading uncommitted data from another transaction |
| Non-Repeatable Read | Re-reading a row returns different data because another transaction updated it |
| Phantom Read | Re-running a query returns different rows because another transaction inserted or deleted rows |

> *MySQL InnoDB default is REPEATABLE READ. Banking systems often use SERIALIZABLE for maximum accuracy.*